

# Лабораторная работа № 5. Построение графиков

---

Абакумова О. М.

Российский университет дружбы народов, Москва, Россия

# Информация

---

- Абакумова Олеся Максимовна
- Студентка
- Российский университет дружбы народов
- 1132220832@pfur.ru
- <https://github.com/omabakumova>



## Цель работы

---

Основная цель работы — освоить синтаксис языка Julia для построения графиков.

## Задание

---

1. Выполнить примеры, приведенные в лабораторной работе.
2. Выполнить задания для самостоятельного выполнения

## Выполнение лабораторной работы

---

Julia поддерживает несколько пакетов для работы с графиками. Использование того или иного пакета зависит от целей, преследуемых пользователем при построении. Стандартным для Julia является пакет `Plots.jl`.

Одним из преимуществ `Plots.jl` является то, что он позволяет легко менять вызов библиотек работы с графикой: `(gr (), pyplot (), plotlyjs ())`.

Далее рассмотрим построение графика функции

$f(x) = (3x^2 + 6x - 9)e^{-0.3x}$  разными способами. Фактически для построения графика функции требуется иметь массив соответствующих значений  $x$  и  $y$ .

# Предварительные сведения

```
In [2]: # задание функции:
f(x) = (3x.^2 + 6x - 9).*exp.(-0.3x)

Out[2]: f (generic function with 1 method)

In [3]: # генерирование массива значений x в диапазоне от -5 до 10 с шагом 0,1
# (шаг задан через указание длины массива):
x = collect(range(-5,10,length=151))

Out[3]: 151-element Vector{Float64}:
 -5.0
 -4.9
 -4.8
 -4.7
 -4.6
 -4.5
 -4.4
 -4.3
 -4.2
 -4.1
 -4.0
 -3.9
 -3.8
  ⋮
  8.9
  9.0
  9.1
  9.2
  9.3
  9.4
  9.5
  9.6
  9.7
  9.8
  9.9
 10.0

In [4]: # генерирование массива значений y:
y = f(x)

Out[4]: 151-element Vector{Float64}:
 161.34080653217032
 146.26477779394003
 132.19219298833204
 118.06033333333333
 103.93866666666667
 89.81700000000001
 75.69533333333333
 61.57366666666667
 47.45200000000001
 33.33033333333333
 19.20866666666667
 5.087000000000001
 -10.034666666666667
 -25.15633333333333
 -40.27800000000001
 -55.39966666666667
 -70.52133333333333
 -85.64300000000001
 -100.76466666666667
 -115.88633333333333
 -131.00800000000001
 -146.12966666666667
 -161.25133333333333
 -176.37300000000001
 -191.49466666666667
 -206.61633333333333
 -221.73800000000001
 -236.85966666666667
 -251.98133333333333
 -267.10300000000001
 -282.22466666666667
 -297.34633333333333
 -312.46800000000001
 -327.58966666666667
 -342.71133333333333
 -357.83300000000001
 -372.95466666666667
 -388.07633333333333
 -403.19800000000001
 -418.31966666666667
 -433.44133333333333
 -448.56300000000001
 -463.68466666666667
 -478.80633333333333
 -493.92800000000001
 -509.04966666666667
 -524.17133333333333
 -539.29300000000001
 -554.41466666666667
 -569.53633333333333
 -584.65800000000001
 -599.77966666666667
 -614.90133333333333
 -630.02300000000001
 -645.14466666666667
 -660.26633333333333
 -675.38800000000001
 -690.50966666666667
 -705.63133333333333
 -720.75300000000001
 -735.87466666666667
 -750.99633333333333
 -766.11800000000001
 -781.23966666666667
 -796.36133333333333
 -811.48300000000001
 -826.60466666666667
 -841.72633333333333
 -856.84800000000001
 -871.96966666666667
 -887.09133333333333
 -902.21300000000001
 -917.33466666666667
 -932.45633333333333
 -947.57800000000001
 -962.69966666666667
 -977.82133333333333
 -992.94300000000001
 -1008.06466666666667
 -1023.18633333333333
 -1038.30800000000001
 -1053.42966666666667
 -1068.55133333333333
 -1083.67300000000001
 -1098.79466666666667
 -1113.91633333333333
 -1129.03800000000001
 -1144.15966666666667
 -1159.28133333333333
 -1174.40300000000001
 -1189.52466666666667
 -1204.64633333333333
 -1219.76800000000001
 -1234.88966666666667
 -1250.01133333333333
 -1265.13300000000001
 -1280.25466666666667
 -1295.37633333333333
 -1310.49800000000001
 -1325.61966666666667
 -1340.74133333333333
 -1355.86300000000001
 -1370.98466666666667
 -1386.10633333333333
 -1401.22800000000001
 -1416.34966666666667
 -1431.47133333333333
 -1446.59300000000001
 -1461.71466666666667
 -1476.83633333333333
 -1491.95800000000001
 -1507.07966666666667
 -1522.20133333333333
 -1537.32300000000001
 -1552.44466666666667
 -1567.56633333333333
 -1582.68800000000001
 -1597.80966666666667
 -1612.93133333333333
 -1628.05300000000001
 -1643.17466666666667
 -1658.29633333333333
 -1673.41800000000001
 -1688.53966666666667
 -1703.66133333333333
 -1718.78300000000001
 -1733.90466666666667
 -1749.02633333333333
 -1764.14800000000001
 -1779.26966666666667
 -1794.39133333333333
 -1809.51300000000001
 -1824.63466666666667
 -1839.75633333333333
 -1854.87800000000001
 -1869.99966666666667
 -1885.12133333333333
 -1900.24300000000001
 -1915.36466666666667
 -1930.48633333333333
 -1945.60800000000001
 -1960.72966666666667
 -1975.85133333333333
 -1990.97300000000001
 -2006.09466666666667
 -2021.21633333333333
 -2036.33800000000001
 -2051.45966666666667
 -2066.58133333333333
 -2081.70300000000001
 -2096.82466666666667
 -2111.94633333333333
 -2127.06800000000001
 -2142.18966666666667
 -2157.31133333333333
 -2172.43300000000001
 -2187.55466666666667
 -2202.67633333333333
 -2217.79800000000001
 -2232.91966666666667
 -2248.04133333333333
 -2263.16300000000001
 -2278.28466666666667
 -2293.40633333333333
 -2308.52800000000001
 -2323.64966666666667
 -2338.77133333333333
 -2353.89300000000001
 -2369.01466666666667
 -2384.13633333333333
 -2399.25800000000001
 -2414.37966666666667
 -2429.50133333333333
 -2444.62300000000001
 -2459.74466666666667
 -2474.86633333333333
 -2489.98800000000001
 -2505.10966666666667
 -2520.23133333333333
 -2535.35300000000001
 -2550.47466666666667
 -2565.59633333333333
 -2580.71800000000001
 -2595.83966666666667
 -2610.96133333333333
 -2626.08300000000001
 -2641.20466666666667
 -2656.32633333333333
 -2671.44800000000001
 -2686.56966666666667
 -2701.69133333333333
 -2716.81300000000001
 -2731.93466666666667
 -2747.05633333333333
 -2762.17800000000001
 -2777.29966666666667
 -2792.42133333333333
 -2807.54300000000001
 -2822.66466666666667
 -2837.78633333333333
 -2852.90800000000001
 -2868.02966666666667
 -2883.15133333333333
 -2898.27300000000001
 -2913.39466666666667
 -2928.51633333333333
 -2943.63800000000001
 -2958.75966666666667
 -2973.88133333333333
 -2989.00300000000001
 -3004.12466666666667
 -3019.24633333333333
 -3034.36800000000001
 -3049.48966666666667
 -3064.61133333333333
 -3079.73300000000001
 -3094.85466666666667
 -3109.97633333333333
 -3125.09800000000001
 -3140.21966666666667
 -3155.34133333333333
 -3170.46300000000001
 -3185.58466666666667
 -3200.70633333333333
 -3215.82800000000001
 -3230.94966666666667
 -3246.07133333333333
 -3261.19300000000001
 -3276.31466666666667
 -3291.43633333333333
 -3306.55800000000001
 -3321.67966666666667
 -3336.80133333333333
 -3351.92300000000001
 -3367.04466666666667
 -3382.16633333333333
 -3397.28800000000001
 -3412.40966666666667
 -3427.53133333333333
 -3442.65300000000001
 -3457.77466666666667
 -3472.89633333333333
 -3488.01800000000001
 -3503.13966666666667
 -3518.26133333333333
 -3533.38300000000001
 -3548.50466666666667
 -3563.62633333333333
 -3578.74800000000001
 -3593.86966666666667
 -3608.99133333333333
 -3624.11300000000001
 -3639.23466666666667
 -3654.35633333333333
 -3669.47800000000001
 -3684.59966666666667
 -3699.72133333333333
 -3714.84300000000001
 -3729.96466666666667
 -3745.08633333333333
 -3760.20800000000001
 -3775.32966666666667
 -3790.45133333333333
 -3805.57300000000001
 -3820.69466666666667
 -3835.81633333333333
 -3850.93800000000001
 -3866.05966666666667
 -3881.18133333333333
 -3896.30300000000001
 -3911.42466666666667
 -3926.54633333333333
 -3941.66800000000001
 -3956.78966666666667
 -3971.91133333333333
 -3987.03300000000001
 -4002.15466666666667
 -4017.27633333333333
 -4032.39800000000001
 -4047.51966666666667
 -4062.64133333333333
 -4077.76300000000001
 -4092.88466666666667
 -4108.00633333333333
 -4123.12800000000001
 -4138.24966666666667
 -4153.37133333333333
 -4168.49300000000001
 -4183.61466666666667
 -4198.73633333333333
 -4213.85800000000001
 -4228.97966666666667
 -4244.10133333333333
 -4259.22300000000001
 -4274.34466666666667
 -4289.46633333333333
 -4304.58800000000001
 -4319.70966666666667
 -4334.83133333333333
 -4349.95300000000001
 -4365.07466666666667
 -4380.19633333333333
 -4395.31800000000001
 -4410.43966666666667
 -4425.56133333333333
 -4440.68300000000001
 -4455.80466666666667
 -4470.92633333333333
 -4486.04800000000001
 -4501.16966666666667
 -4516.29133333333333
 -4531.41300000000001
 -4546.53466666666667
 -4561.65633333333333
 -4576.77800000000001
 -4591.89966666666667
 -4607.02133333333333
 -4622.14300000000001
 -4637.26466666666667
 -4652.38633333333333
 -4667.50800000000001
 -4682.62966666666667
 -4697.75133333333333
 -4712.87300000000001
 -4727.99466666666667
 -4743.11633333333333
 -4758.23800000000001
 -4773.35966666666667
 -4788.48133333333333
 -4803.60300000000001
 -4818.72466666666667
 -4833.84633333333333
 -4848.96800000000001
 -4864.08966666666667
 -4879.21133333333333
 -4894.33300000000001
 -4909.45466666666667
 -4924.57633333333333
 -4939.69800000000001
 -4954.81966666666667
 -4969.94133333333333
 -4985.06300000000001
 -5000.18466666666667
 -5015.30633333333333
 -5030.42800000000001
 -5045.54966666666667
 -5060.67133333333333
 -5075.79300000000001
 -5090.91466666666667
 -5106.03633333333333
 -5121.15800000000001
 -5136.27966666666667
 -5151.40133333333333
 -5166.52300000000001
 -5181.64466666666667
 -5196.76633333333333
 -5211.88800000000001
 -5227.00966666666667
 -5242.13133333333333
 -5257.25300000000001
 -5272.37466666666667
 -5287.49633333333333
 -5302.61800000000001
 -5317.73966666666667
 -5332.86133333333333
 -5347.98300000000001
 -5363.10466666666667
 -5378.22633333333333
 -5393.34800000000001
 -5408.46966666666667
 -5423.59133333333333
 -5438.71300000000001
 -5453.83466666666667
 -5468.95633333333333
 -5484.07800000000001
 -5499.19966666666667
 -5514.32133333333333
 -5529.44300000000001
 -5544.56466666666667
 -5559.68633333333333
 -5574.80800000000001
 -5589.92966666666667
 -5605.05133333333333
 -5620.17300000000001
 -5635.29466666666667
 -5650.41633333333333
 -5665.53800000000001
 -5680.65966666666667
 -5695.78133333333333
 -5710.90300000000001
 -5726.02466666666667
 -5741.14633333333333
 -5756.26800000000001
 -5771.38966666666667
 -5786.51133333333333
 -5801.63300000000001
 -5816.75466666666667
 -5831.87633333333333
 -5846.99800000000001
 -5862.11966666666667
 -5877.24133333333333
 -5892.36300000000001
 -5907.48466666666667
 -5922.60633333333333
 -5937.72800000000001
 -5952.84966666666667
 -5967.97133333333333
 -5983.09300000000001
 -5998.21466666666667
 -6013.33633333333333
 -6028.45800000000001
 -6043.57966666666667
 -6058.70133333333333
 -6073.82300000000001
 -6088.94466666666667
 -6104.06633333333333
 -6119.18800000000001
 -6134.30966666666667
 -6149.43133333333333
 -6164.55300000000001
 -6179.67466666666667
 -6194.79633333333333
 -6209.91800000000001
 -6225.03966666666667
 -6240.16133333333333
 -6255.28300000000001
 -6270.40466666666667
 -6285.52633333333333
 -6300.64800000000001
 -6315.76966666666667
 -6330.89133333333333
 -6346.01300000000001
 -6361.13466666666667
 -6376.25633333333333
 -6391.37800000000001
 -6406.49966666666667
 -6421.62133333333333
 -6436.74300000000001
 -6451.86466666666667
 -6466.98633333333333
 -6482.10800000000001
 -6497.22966666666667
 -6512.35133333333333
 -6527.47300000000001
 -6542.59466666666667
 -6557.71633333333333
 -6572.83800000000001
 -6587.95966666666667
 -6603.08133333333333
 -6618.20300000000001
 -6633.32466666666667
 -6648.44633333333333
 -6663.56800000000001
 -6678.68966666666667
 -6693.81133333333333
 -6708.93300000000001
 -6724.05466666666667
 -6739.17633333333333
 -6754.29800000000001
 -6769.41966666666667
 -6784.54133333333333
 -6799.66300000000001
 -6814.78466666666667
 -6829.90633333333333
 -6845.02800000000001
 -6860.14966666666667
 -6875.27133333333333
 -6890.39300000000001
 -6905.51466666666667
 -6920.63633333333333
 -6935.75800000000001
 -6950.87966666666667
 -6965.99133333333333
 -6981.11300000000001
 -6996.23466666666667
 -7011.35633333333333
 -7026.47800000000001
 -7041.59966666666667
 -7056.72133333333333
 -7071.84300000000001
 -7086.96466666666667
 -7102.08633333333333
 -7117.20800000000001
 -7132.32966666666667
 -7147.45133333333333
 -7162.57300000000001
 -7177.69466666666667
 -7192.81633333333333
 -7207.93800000000001
 -7223.05966666666667
 -7238.18133333333333
 -7253.30300000000001
 -7268.42466666666667
 -7283.54633333333333
 -7298.66800000000001
 -7313.78966666666667
 -7328.91133333333333
 -7344.03300000000001
 -7359.15466666666667
 -7374.27633333333333
 -7389.39800000000001
 -7404.51966666666667
 -7419.64133333333333
 -7434.76300000000001
 -7449.88466666666667
 -7465.00633333333333
 -7480.12800000000001
 -7495.24966666666667
 -7510.37133333333333
 -7525.49300000000001
 -7540.61466666666667
 -7555.73633333333333
 -7570.85800000000001
 -7585.97966666666667
 -7601.10133333333333
 -7616.22300000000001
 -7631.34466666666667
 -7646.46633333333333
 -7661.58800000000001
 -7676.70966666666667
 -7691.83133333333333
 -7706.95300000000001
 -7722.07466666666667
 -7737.19633333333333
 -7752.31800000000001
 -7767.43966666666667
 -7782.56133333333333
 -7797.68300000000001
 -7812.80466666666667
 -7827.92633333333333
 -7843.04800000000001
 -7858.16966666666667
 -7873.29133333333333
 -7888.41300000000001
 -7903.53466666666667
 -7918.65633333333333
 -7933.77800000000001
 -7948.89966666666667
 -7964.02133333333333
 -7979.14300000000001
 -7994.26466666666667
 -8009.38633333333333
 -8024.50800000000001
 -8039.62966666666667
 -8054.75133333333333
 -8069.87300000000001
 -8084.99466666666667
 -8100.11633333333333
 -8115.23800000000001
 -8130.35966666666667
 -8145.48133333333333
 -8160.60300000000001
 -8175.72466666666667
 -8190.84633333333333
 -8205.96800000000001
 -8221.08966666666667
 -8236.21133333333333
 -8251.33300000000001
 -8266.45466666666667
 -8281.57633333333333
 -8296.69800000000001
 -8311.81966666666667
 -8326.94133333333333
 -8342.06300000000001
 -8357.18466666666667
 -8372.30633333333333
 -8387.42800000000001
 -8402.54966666666667
 -8417.67133333333333
 -8432.79300000000001
 -8447.91466666666667
 -8463.03633333333333
 -8478.15800000000001
 -8493.27966666666667
 -8508.40133333333333
 -8523.52300000000001
 -8538.644
```

# Предварительные сведения

```
In [5]: # указывается, что для построения графика используется gr():  
gr()
```

```
Out[5]: Plots.GRBackend()
```

```
In [6]: # задание опций при построении графика  
# (название кривой, подписи по осям, цвет графика):  
plot(x,y,  
      title="A simple curve",  
      xlabel="Variable x",  
      ylabel="Variable y",  
      color="blue")
```

```
Out[6]:
```

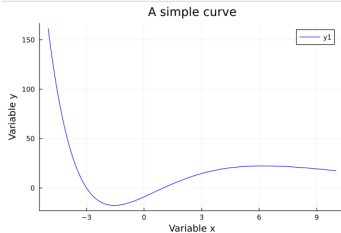


Рис. 2: График функции  $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$ , построенный при помощи `gr()`

# Предварительные сведения

```
In [7]: # указывается, что для построения графика используется pyplot():
pyplot()
# задание опций при построении графика
# (название кривой, подписи по осям, цвет графика):
plot(x,y,
      title="Простая кривая",
      xlabel="Переменная x",
      ylabel="Переменная y",
      color="blue")
```

Out[7]:

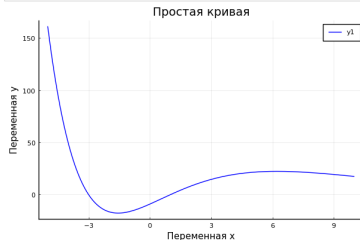


Рис. 3: График функции  $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$ , построенный при помощи pyplot()

# Предварительные сведения

```
In [17]: # указывается, что для построения графика используется pyplot():
plotly()
# задание опций при построении графика
# (название кривой, подписи по осям, цвет графика):
plot(x,y,
      title="Простая кривая",
      xlabel="Переменная x",
      ylabel="Переменная y",
      color="blue")
```

Out[17]:

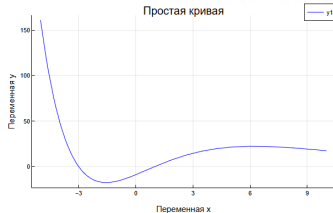


Рис. 4: График функции  $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$ , построенный при помощи plotly()

# Предварительные сведения

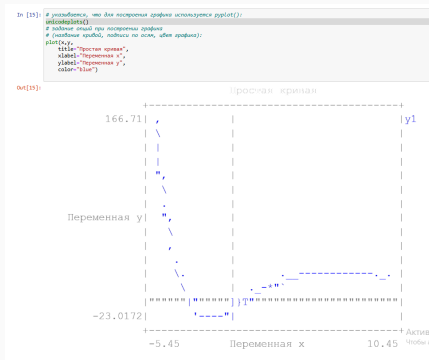


Рис. 5: График функции  $f(x) = (3x^2 + 6x - 9)e^{-0.3x}$ , построенный при помощи `unicodeplots()`

Далее на примере графика функции  $\sin(x)$  и графика разложения этой функции в ряд Тейлора:

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n+1)!}, \quad x \in \mathbb{C}$$

рассмотрим дополнительные возможности пакетов для работы с графикой.

# Опции при построении графика

```
In [19]: # указывается, что для построения графика используется pyplot():  
         pyplot()  
         # Задан функция:  
         # задание функции sin(x):  
         sin_theor(x) = sin(x)  
         # Строим график функции (рис. 5.3):  
         # построение графика функции sin(x):  
         plot(sin_theor)
```

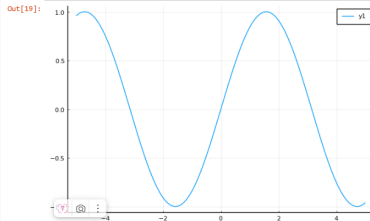


Рис. 6: График функции  $\sin(x)$

# Опции при построении графика

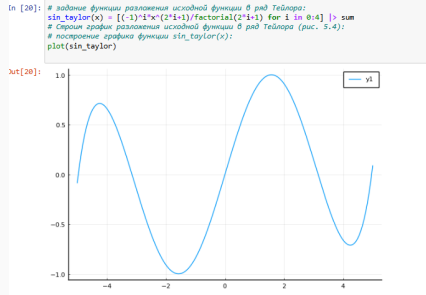


Рис. 7: График функции разложения исходной функции в ряд Тейлора

# Опции при построении графика

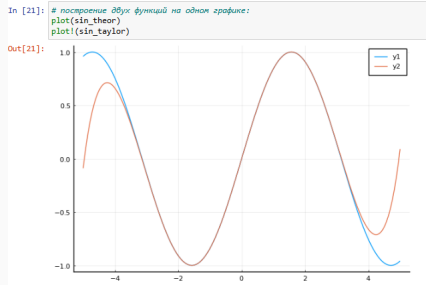


Рис. 8: Графики исходной функции и её разложения в ряд Тейлора

# Опции при построении графика

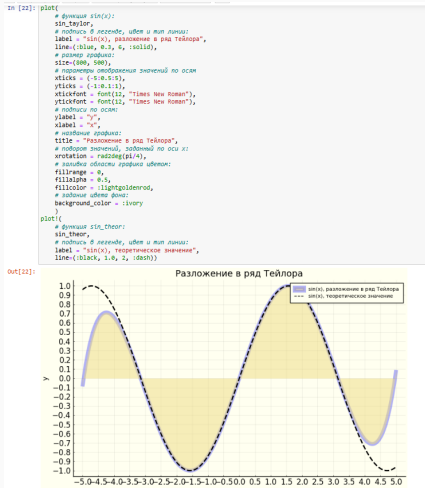


Рис. 9: Вид графиков после добавления опций при их построении

```
In [23]: # сохранение графика в файле в формате pdf или png:  
savefig("taylor.pdf")  
savefig("taylor.png")  
Out[23]: "C:\\Users\\user\\Downloads\\taylor.png"
```

Рис. 10: Сохранение построенного графика

Графики в виде точек на плоскости или в пространстве часто используются в статистических исследованиях.

# Точечный график

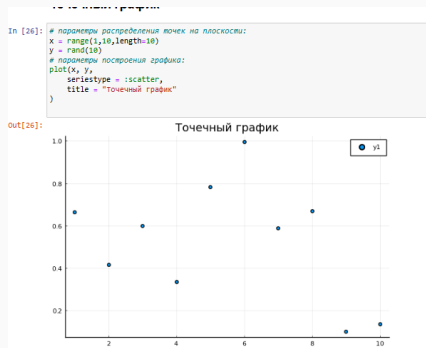


Рис. 11: График десяти случайных значений на плоскости

# Точечный график

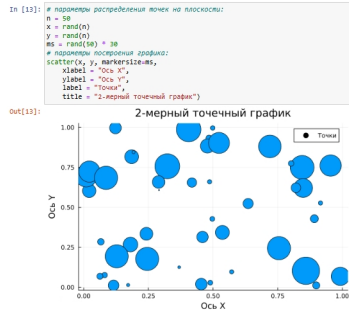


Рис. 12: График пятидесяти случайных значений на плоскости с различными опциями отображения

# Точечный график

```
In [33]: # параметры распределения точек в пространстве:
```

```
n = 50
```

```
x = rand(n)
```

```
y = rand(n)
```

```
z = rand(n)
```

```
ms = rand(50) * 30
```

```
# параметры построения графика:
```

```
scatter(x, y, z, markersize=ms,
```

```
        xlabel = "Ось X",
```

```
        ylabel = "Ось Y",
```

```
        zlabel = "Ось Z",
```

```
        label = "Точки",
```

```
        title = "3-мерный точечный график")
```

```
Out[33]:
```

3-мерный точечный график

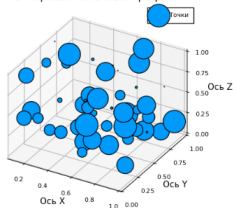


Рис. 13: График пятидесяти случайных значений в пространстве с различными опциями отображения

Аппроксимация — научный метод, состоящий в замене объектов их более простыми аналогами, сходными по своим свойствам. Пусть на некотором отрезке в точках  $(x_0, x_1, x_2, \dots, x_N)$  известны значения некоторой функции  $f(x)$ , а именно  $(y_0, y_1, y_2, \dots, y_N)$ .

Требуется определить параметры  $a_i$  многочлена вида  $F(x) = a_0 + a_1x + \dots + a_kx^k$ , где  $k < N$  такое, что сумма квадратов отклонений значений  $y$  от значений функции  $F(y)$  в заданных точках  $x$  минимальна, т.е.

$$S = \sum_{i=0}^N [y_i - F(x_i, a_0, a_1, \dots, a_k)]^2 \rightarrow \min$$

Геометрически это означает, что нужно найти кривую  $y = F(x)$ , полином которой проходит как можно ближе к каждой из заданных точек. Такая задача может быть решена, если решить систему уравнений вида:

$$A\vec{x} = \vec{y},$$

где  $A$  — матрица коэффициентов многочлена  $F(x)$ , а  $\vec{x}$  и  $\vec{y}$  — векторы соответствующих значений. Для демонстрации зададим искусственно некоторую функцию, в данном случае похожую по поведению на экспоненту

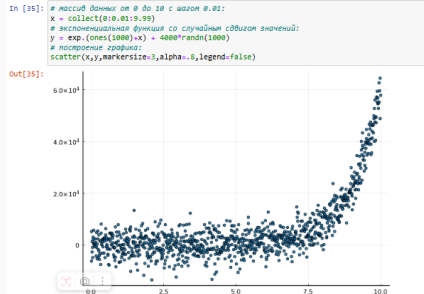


Рис. 14: Пример функции

```
In [38]: # определение массива для нахождения коэффициентов полинома:
A = [ones(1000) x x.^2 x.^3 x.^4 x.^5]
# решение матричного уравнения:
c = A\y
# построение полинома (используем другое имя переменной):
f_poly = c[1]*ones(1000) + c[2]*x + c[3]*x.^2 + c[4]*x.^3 + c[5]*x.^4 + c[6]*x.^5
# построение графика аппроксимирующей функции:
plot(x, f_poly, linewidth=3, color='red')
```

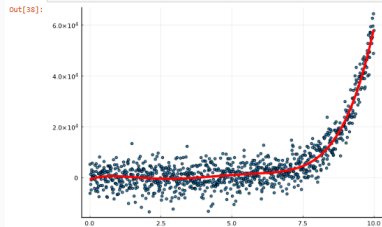


Рис. 15: Пример аппроксимации исходной функции полиномом 5-й степени

# Две оси ординат

```
In [6]: # пример случайной траектории  
# (заданы обозначение траектории, легенда вверху справа, без сетки)  
plot(randn(100),  
     ylabel="y1",  
     leg="topright",  
     grid="off",  
     )
```

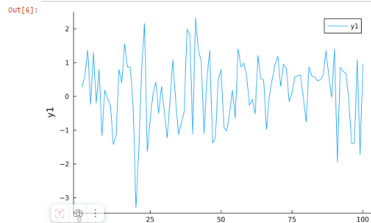


Рис. 16: Пример отдельно построенной траектории

# Две оси ординат

```
In [7]: # пример добавления на график второй случайной траектории
# (задано обозначение траектории и её цвет, легенда снизу справа, без сетки)
# задана рамка графика
plot(twinx(), randn(100)*10,
     c='red',
     ylabel='y2',
     leg=:bottomright,
     grid=:off,
     box=:on,
     # size=(600, 400)
)
```

Out[7]:

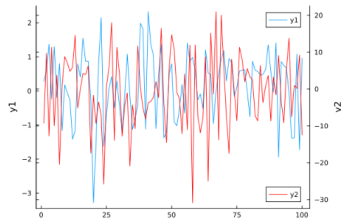


Рис. 17: Пример двух траекторий на одном графике с двумя осями ординат

Приведём пример построения графика функции

$$r(\vartheta) = 1 + \cos \vartheta \cdot \sin^2 \vartheta$$

в полярных координатах.

# Полярные координаты

```
In [8]: # функция в полярных координатах:  
r(θ) = 1 + cos(θ) * sin(θ)^2  
# полярная система координат:  
θ = range(0, stop=2π, length=50)  
# график функции, заданной в полярных координатах:  
plot(θ, r.(θ),  
      proj='polar',  
      lims=(0,1.5)  
)
```

Out[8]:

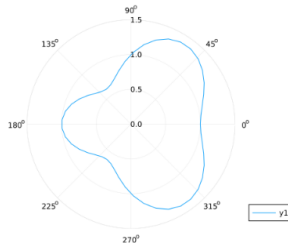


Рис. 18: График функции, заданной в полярных координатах

При параметрическом представлении графика некоторой функции координаты на графике задаются как функции от некоторого набора свободных параметров. В случае одного параметра получим параметрическое уравнение кривой. Выражая координаты точек поверхности через два свободных параметра, получим параметрическое задание поверхности.

Приведём пример построения графика параметрически заданной кривой на плоскости. Кривая задана выражениями

$$\begin{aligned}x(t) &= \sin(t), \\ y(t) &= \sin(2t),\end{aligned} \quad 0 \leq t \leq 2\pi$$

# Параметрический график

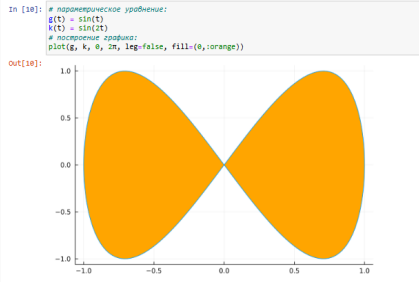


Рис. 19: Параметрический график кривой на плоскости

Приведём пример построения графика параметрически заданной кривой в пространстве. Кривая задана выражениями:

$$x(t) = \cos(t), \quad y(t) = \sin(t), \quad z(t) = \sin(5t).$$

# Параметрический график

```
In [12]: t = range(0, stop=10, length=1000)
         g_val = cos.(t)
         k_val = sin.(t)
         p_val = sin.(5t)
         plot(g_val, k_val, p_val)
```

Out[12]:

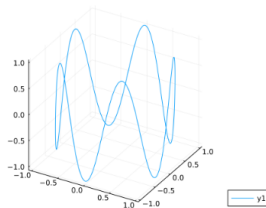


Рис. 20: Параметрический график кривой в пространстве

Для построения поверхности, заданной уравнением  $f(x, y) = x^2 + y^2$ , можно воспользоваться функцией `surface()`

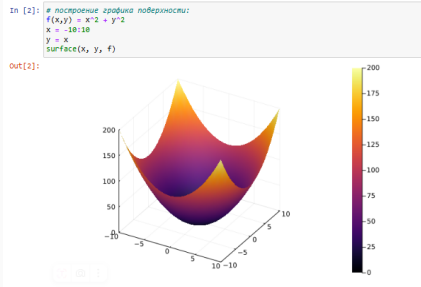


Рис. 21: График поверхности (использована функция `surface()`)

# График поверхности

```
In [3]: # построение графика поверхности:  
f(x,y) = x^2 + y^2  
x = -10:10  
y = x  
plot(x, y, f,  
      linestyle='wireframe'  
)
```

Out[3]:

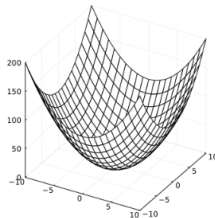


Рис. 22: График поверхности (использована функция plot())

# График поверхности

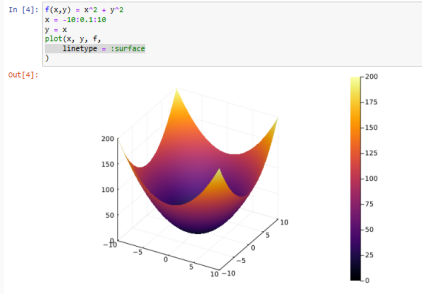


Рис. 23: Сглаженный график поверхности

# График поверхности

```
In [9]: x=range(-2,stop=2,length=100)  
y=range(sqrt(2),stop=2,length=100)  
f(x,y) = x*y-x-y+1  
plot(X,Y,f,  
      linestyle = :surface,  
      c=cgrad([:red,:blue]),  
      camera=(-30,30),  
      )
```

Out[9]:

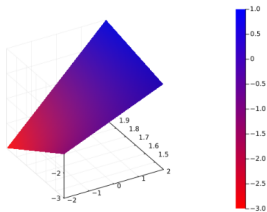


Рис. 24: График поверхности с изменённым углом зрения

Линией уровня некоторой функции от двух переменных называется множество точек на координатной плоскости, в которых функция принимает одинаковые значения. Линий уровня бесконечно много, и через каждую точку области определения можно провести линию уровня.

С помощью линий уровня можно определить наибольшее и наименьшее значение исходной функции от двух переменных. Каждая из этих линий соответствует определённому значению высоты.

Поверхности уровня представляют собой непересекающиеся пространственные поверхности.

$$g(x, y) = (3x + y^2) |\sin(x) + \cos(y)|$$

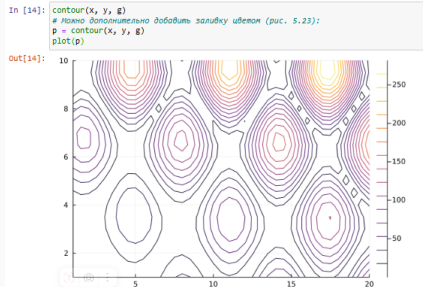


Рис. 25: Линии уровня

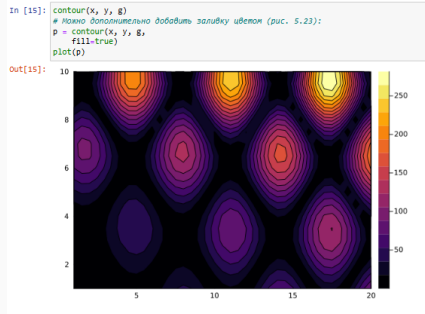


Рис. 26: Линии уровня с заполнением

Если каждой точке некоторой области пространства поставлен в соответствие вектор с началом в данной точке, то говорят, что в этой области задано векторное поле.

Векторные поля задают векторными функциями. Для функции  $h(x, y) = x^3 - 3x + y^2$  сначала построим её график и линии уровня.

```
In [45]: # определение переменных:  
X = range(-2, stop=2, length=100)  
Y = range(-2, stop=2, length=100)  
# определение функции:  
h(x, y) = x**3 - 3x + y**2
```

```
Out[45]: h (generic function with 1 method)
```

```
In [46]: # построение поверхности:  
plot(X,Y,h,  
      linestyle = :surface  
)
```

```
Out[46]:
```

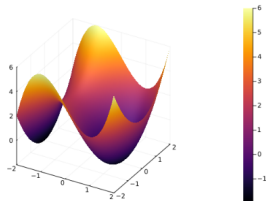


Рис. 27: График функции  $h(x, y) = x^3 - 3x + y^2$

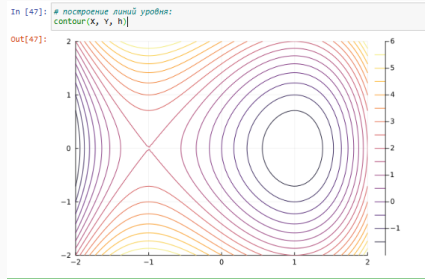


Рис. 28: Линии уровня для функции  $h(x, y) = x^3 - 3x + y^2$

Векторное поле можно охарактеризовать векторными линиями. Каждая точка векторной линии является началом вектора поля, который лежит на касательной в данной точке.

Для заданной функции построим векторное поле.

# Векторные поля

```
In [50]: h(x, y) = x**3 - 3x + y**2
dh(x, y) = ((3x**2 - 3)/25, (2y)/25) # КОПЕКА, не вектор!

p = contour(x, y, h, levels=20, fill=False)

# Сетка для стрелок (меньше точек)
xs = range(-2, stop=2, length=12)
ys = range(-2, stop=2, length=12)
Xq = vec([x for x in xs, y in ys])
Yq = vec([y for x in xs, y in ys])

quiver!(Xq, Yq, quiver=dh, color=:blue)
display(p)
```

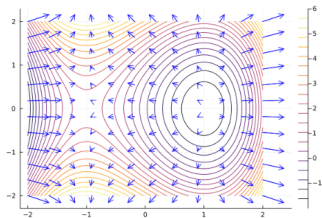


Рис. 29: Векторное поле функции  $h(x, y) = x^3 - 3x + y^2$

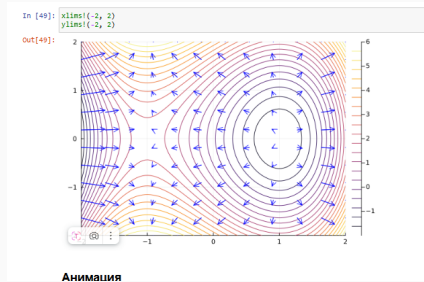


Рис. 30: Векторное поле функции  $h(x, y) = x^3 - 3x + y^2$  (скорректирована область видимости)

Технически анимированное изображение представляет собой несколько наложенных изображений (или построенных в разных точках графиках) в одном файле.

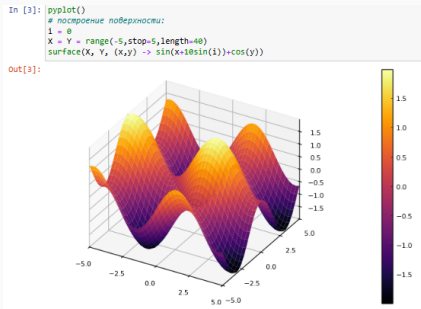
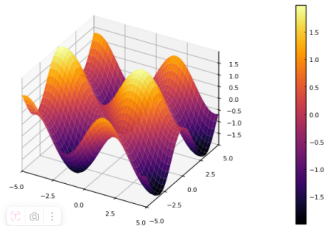
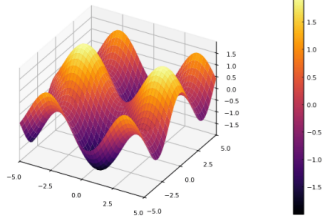


Рис. 31: Статичный график поверхности

```
In [4]: # анимация:  
X = Y = range(-5,stop=5,length=40)  
@gif for i in range(0,stop=20,length=100)  
    surface(X, Y, (x,y) -> sin(x+10sin(i))+cos(y))  
end  
[ Info: Saved animation to c:\Users\user\Downloads\tmp.gif
```



Out[4]:



Гипоциклоида — плоская кривая, образуемая точкой окружности, катящейся по внутренней стороне другой окружности без скольжения:

$$\begin{aligned}x &= r(k-1) \left( \cos t + \frac{\cos((k-1)t)}{k-1} \right), \\y &= r(k-1) \left( \sin t - \frac{\sin((k-1)t)}{k-1} \right),\end{aligned}$$

где  $k = \frac{R}{r}$ ,  $R$  — радиус неподвижной окружности,  $r$  — радиус катящейся окружности. Модуль величины  $k$  определяет форму гипоциклоиды.

```
In [5]: # радиус малой окружности:
r = 1
# коэффициент для построения большой окружности:
k = 3
# число отсчётов:
n = 100
```

Out[5]: 100

```
In [7]: # массив значений угла  $\theta$ :
# theta from 0 to 2pi ( + a little extra)
 $\theta$  = collect(0:2*pi/100:2*pi+2*pi/100)
# массивы значений координат:
X = r*k*cos.( $\theta$ )
Y = r*k*sin.( $\theta$ )
```

```
Out[7]: 101-element Vector{Float64}:
 0.0
 0.18837155858794014
 0.3759997006929128
 0.5621439437571739
 0.7460696614945643
 0.9270509831248421
 1.104373658054034
 1.2773378746952182
 1.445261022305146
 1.60748038493699
 1.7633557568774194
 1.9122719692460692
 2.053641317786066
 |
-1.9122719692460688
-1.7633557568774179
-1.607480384936989
-1.445261022305146
-1.2773378746952166
-1.1043736580540335
-0.9270509831248404
-0.7460696614945634
-0.562143943757174
-0.37599970069291133
-0.1883715585879398
 1.929747179611964e-15
```

# Гипоциклоида

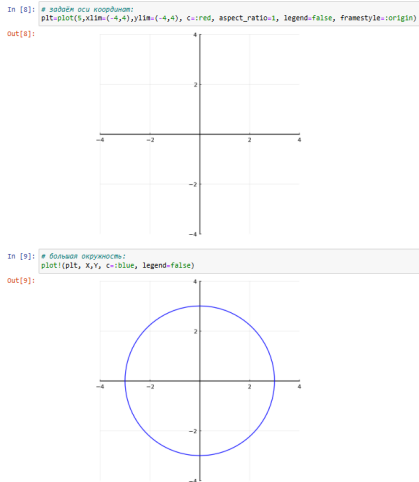
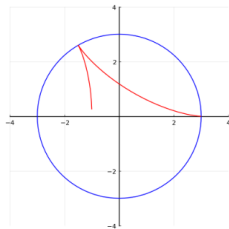


Рис. 34: Большая окружность гипоциклоиды с осями координат

# Гипоциклоида

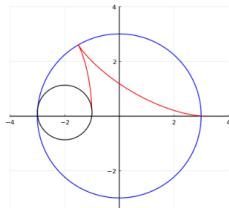
```
In [10]: i = 50  
         t = 0[1:i]  
         # гипоциклоида:  
         x = r*(k-1)*cos.(t) + r*cos.((k-1)*t)  
         y = r*(k-1)*sin.(t) - r*sin.((k-1)*t)  
         plot!(x,y, c=:red)
```

Out[10]:



```
In [11]: # малая окружность:  
         xc = r*(k-1)*cos(t[end]) .+ r*cos.(θ)  
         yc = r*(k-1)*sin(t[end]) .+ r*sin.(θ)  
         plot!(xc,yc,c=:black)
```

Out[11]:



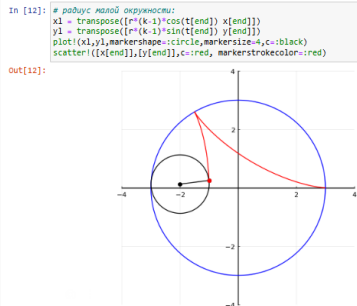


Рис. 36: Малая окружность гипоциклоиды с добавлением радиуса

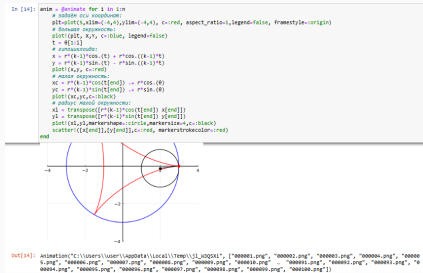


Рис. 37: Анимация движения гипоциклоиды

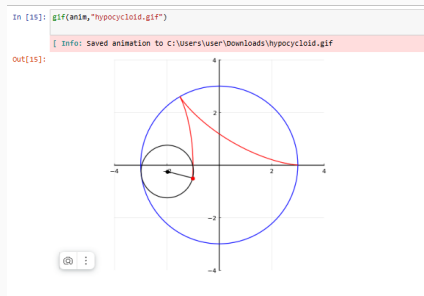


Рис. 38: Анимация движения гипоциклоиды

В исследованиях часто требуется изобразить графики погрешностей измерения.

```
In [16]: # add/remove new/old Statistics:
import Pkg
Pkg.add("Statistics")
using Statistics

Resolving package versions...
Updating `C:\Users\user\.julia\environments\v1.12\Project.toml`
[10740206] + Statistics v1.11.1
Manifest: No packages added to or removed from `C:\Users\user\.julia\environments\v1.12\Manifest.toml`

In [17]: sds = [1, 1/2, 1/4, 1/8, 1/16, 1/32]

Out[17]: 6-element Vector{Float64}:
 1.0
 0.5
 0.25
 0.125
 0.0625
 0.03125

In [18]: n = 10
y = [mean(sd*randn(n)) for sd in sds]
errs = 1.96 * sds / sqrt(n)

Out[18]: 6-element Vector{Float64}:
 0.6190064213930023
 0.30959032180655012
 0.1549516853482506
 0.0774758026741253
 0.03872790113786265
 0.019363950660531323
```

Рис. 39: Задание массива значений и генерация массива ошибок

# Errorbars

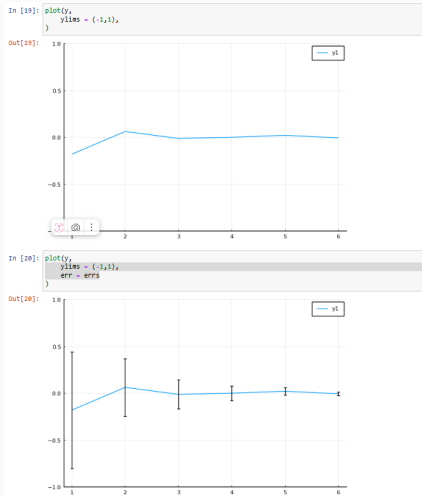


Рис. 40: Графики исходных значений и исходных значений с отклонениями

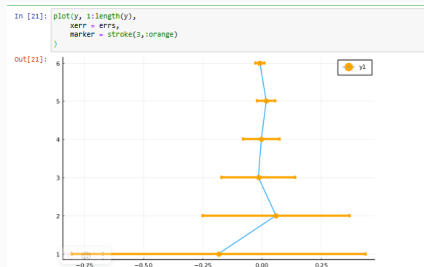


Рис. 41: Поворот графика

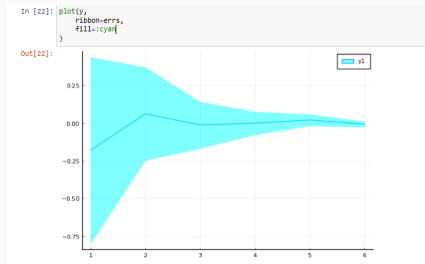


Рис. 42: Заполнение цветом

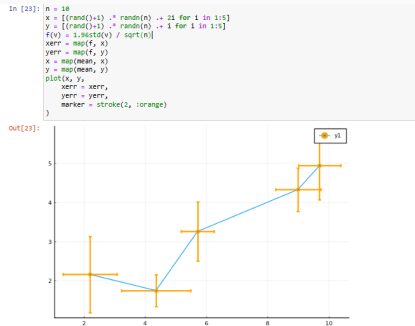


Рис. 43: График ошибок по двум осям

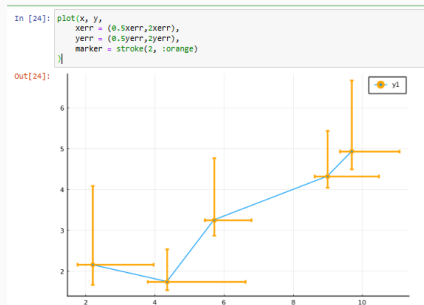


Рис. 44: График асимметричных ошибок по двум осям

# Использование пакета Distributions

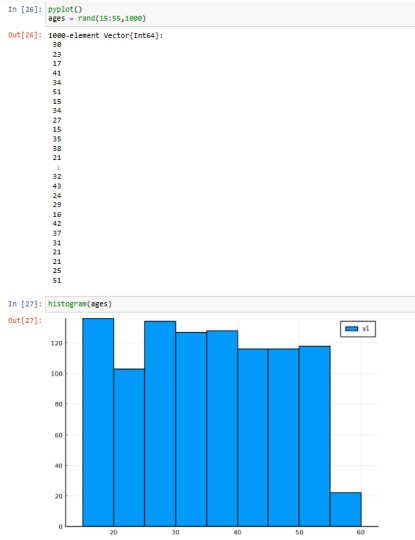


Рис. 15: Гистограмма, построенная по массиву случайных чисел

# Использование пакета Distributions

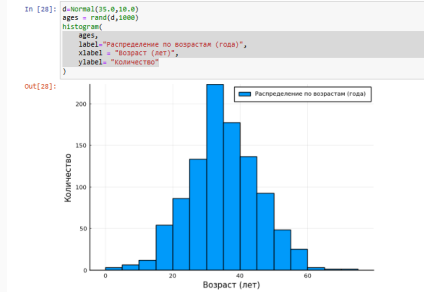


Рис. 46: Гистограмма нормального распределения

# Использование пакета Distributions

```
In [29]: plotly()  
d1=Normal(10.0,5.0);  
d2=Normal(35.0,10.0);  
d3=Normal(60.0,5.0);  
N=1000;  
ages = (Float64[]);  
ages = append!(ages,rand(d1,Int64(ceil(N/2))));  
ages = append!(ages,rand(d2,N));  
ages = append!(ages,rand(d3,Int64(ceil(N/3))));  
histogram(  
    ages,  
    bins=50,  
    label="Распределение по возрастам (года)",  
    xlabel = "Возраст (лет)",  
    ylabel = "количество",  
    title = "Распределение по возрастам (года)"  
)
```

Out[29]:

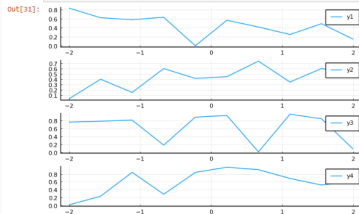


Рис. 47: Гистограмма распределения людей по возрастам

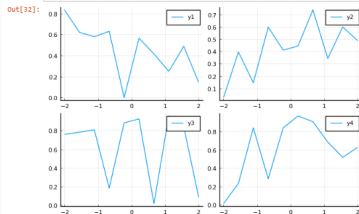
Определим макет расположения графиков. Команда `layout` принимает кортеж `layout = (N, M)`, который строит сетку графиков  $N \times M$ . Например, если задать `layout = (4, 1)` на графике четыре серии, то получим четыре ряда графиков, для автоматического вычисления сетки необходимо передать `layout` целое число

# Подграфики

```
In [31]: # создаем pyplot():  
pyplot()  
# построение серии графиков:  
x = range(-2, 2, length=10)  
y = rand(10, 4)  
plot(x, y,  
      layout=(4, 1)  
)
```



```
In [32]: plot(x, y,  
              layout=4  
)
```



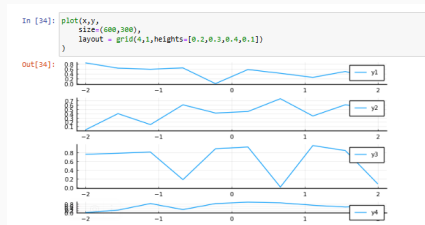


Рис. 49: Серия из 4-х графиков разной высоты в ряд

Аргумент **heights** принимает в качестве входных данных массив с долями желаемых высот. Если в сумме дроби не составляют 1,0, то некоторые подзаголовки могут отображаться неправильно.

```
In [15]: # создание 4 sube subuid:
p1 = plot(x,y)
# график 3 sube почечки:
p2 = scatter(x,y)
# график 3 sube subuid с оформлением:
p3 = plot(x,y[:,1:2],xlabel="labelled plot of two columns",lw=2,title="wide lines")
# 4 subuid subuid:
p4 = histogram(x,y)
plot(
    p1,p2,p3,p4,
    layout=(2,2),
    legend=False,
    size=(800,600),
    background_color = 'ivory'
)
```

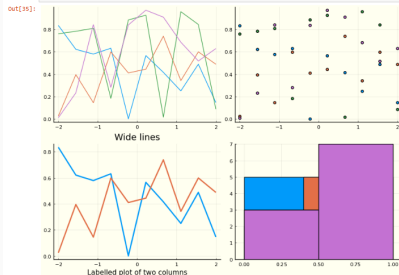


Рис. 50: Объединение нескольких графиков в одной сетке



Рис. 51: Разнообразные варианты представления данных

Применение макроса `@layout` наиболее простой способ определения сложных макетов. Точные размеры могут быть заданы с помощью фигурных скобок, в противном случае пространство будет поровну разделено между графиками

```
In [37]: 1 = @layout [ a{0.3w} [grid(3,3)  
                      b{0.2h} ]]  
plot(  
  rand(10,11),  
  layout = 1, legend = false, seriestype = [:bar :scatter :path],  
  title = ["$j" for j = 1:11, 1:1:11], titleloc = :right, titlefont = font(8)  
)
```

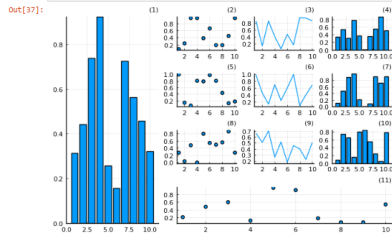


Рис. 52: Демонстрация применения сложного макета для построения графиков

## Задания для самостоятельного выполнения

1. Постройте все возможные типы графиков (простые, точечные, гистограммы и т.д.) функции  $y = \sin(x)$ , где  $x \in [0, 2\pi]$ .  
Отобразите все графики в одном графическом окне

# Задания для самостоятельного выполнения

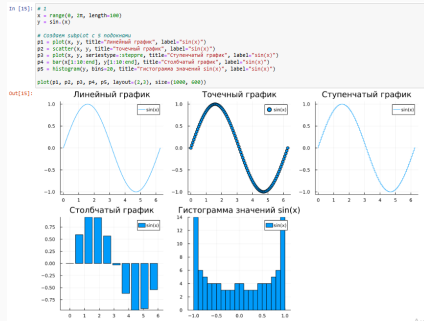


Рис. 53: Все возможные типы графиков функции  $y = \sin(x)$ , где  $x \in [0, 2\pi]$

2. Постройте графики функции  $y = \sin(x)$ , где  $x \in [0, 2\pi]$  со всеми возможными (сколько сможете вспомнить) типами оформления линий графика. Отобразите все графики в одном графическом окне

# Задания для самостоятельного выполнения

```
In [57]: # 2
styles = [:solid, :dash, :dot, :dashdot, :dashdotdot]
colors = [:red, :blue, :green, :orange, :purple]

p = plot()
for (i, style) in enumerate(styles)
    plot(x, sin.(x .+ i*0.1), linestyle=style, linecolor=colors[i], label="Стиль $i")
end

title("sin(x) с разными стилями линий")
display(p)
```

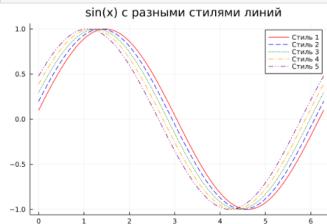


Рис. 54: Демонстрация всех типов линий графика для функции  $y = \sin(x)$ , где  $x \in [0, 2\pi]$

3. Постройте график функции  $y(x) = \pi x^2 \ln(x)$ , назовите оси соответственно. Пусть цвет рамки будет зелёным, а цвет самого графика — красным. Задайте расстояние между надписями и осями так, чтобы надписи полностью уместались в графическом окне. Задайте шрифт надписей. Задайте частоту отметок на осях координат

# Задания для самостоятельного выполнения

```
In [60]: # 3
x = range(0.1, 2, length=100)
y = pi .* x.^2 .* log.(x)

plot(x, y, color=:red, linewidth=2, title="y = πx²ln(x)",
      background_color_outside=:green,
      grid=true,
      xticks=0:0.2:2,
      xguidefont = font(12, "Times New Roman", :white),
      yguidefont = font(12, "Times New Roman", :white),
      xtickfont = font(10, "Times New Roman", :white),
      ytickfont = font(10, "Times New Roman", :white),
      titlefont = font(14, "Times New Roman", :white))
```

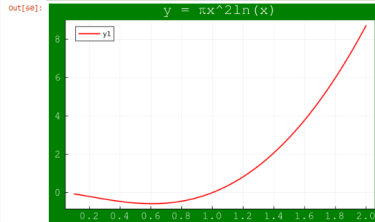


Рис. 55: График функции  $y(x) = \pi x^2 \ln(x)$

## Задания для самостоятельного выполнения

4. Задайте вектор  $x = (-2, -2, 0, 1, 2)$ . В одном графическом окне (в 4-х подокнах) изобразите графически по точкам  $x$  значения функции  $y(x) = x^3 - 3x$  в виде:

- точек,
- линий,
- линий и точек,
- кривой.

Сохраните полученные изображения в файле `figure_familiya.png`, где вместо `familiya` укажите вашу фамилию

# Задания для самостоятельного выполнения

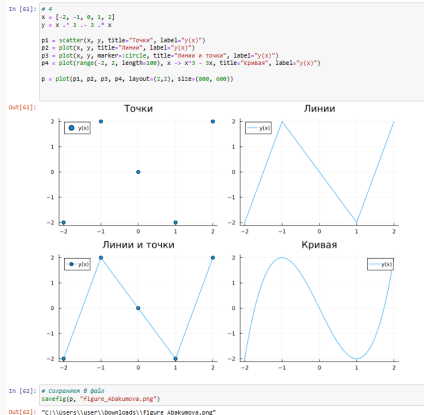


Рис. 56: Графическое изображение по точкам  $x$  значения функции  $y(x) = x^3 - 3x$  в разном виде

5. Задайте вектор  $x = (3, 3.1, 3.2, \dots, 6)$ . Постройте графики функций  $y_1(x) = \pi x$  и  $y_2(x) = \exp(x) \cos(x)$  в указанном диапазоне значений аргумента  $x$  следующим образом:
- постройте оба графика разного цвета на одном рисунке, добавьте легенду и сетку для каждого графика; укажите недостатки у данного построения

# Задания для самостоятельного выполнения

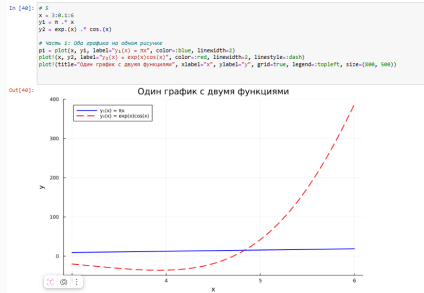


Рис. 57: Первое построение графика

Недостатки первого построения:

- Функции имеют разные масштабы ( $y_1 \sim 10$ ,  $y_2 \sim \pm 200$ ), что делает один график почти незаметным.
- Трудно сравнивать поведение функций из-за разных порядков величин.

# Задания для самостоятельного выполнения

- постройте аналогичный график с двумя осями ординат



Рис. 58: Второе построение графика

6. Постройте график некоторых экспериментальных данных (придумайте сами), учитывая ошибку измерения

# Задания для самостоятельного выполнения

```
In [49]: # 6
# экспериментальные данные
x = 0:0.5:10 # время, сек
y = 20 + 0.5 * sin(x) + 0.3 * cos(2x) # модельная температура
err = fill(0.5, length(x)) # ошибка измерения ±0.5

p = plot(x, y, yerror=err,
         title="экспериментальные данные с ошибкой",
         xlabel="Время (сек)", ylabel="Температура (°C)",
         label="измеренные значения",
         marker=:circle, color=:blue, linewidth=2,
         grid=true, legend=:topright)
```

Out[49]:

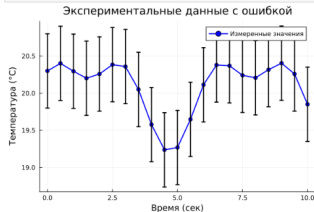


Рис. 59: График экспериментальных данных с учетом ошибки измерения

7. Постройте точечный график случайных данных. Подпишите оси, легенду, название графика

# Задания для самостоятельного выполнения

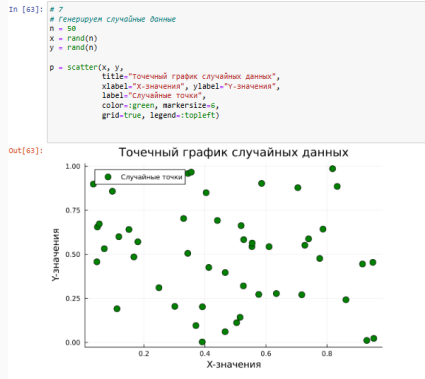


Рис. 60: Точечный график случайных данных

8. Постройте 3-мерный точечный график случайных данных. Подпишите оси, легенду, название графика

# Задания для самостоятельного выполнения

```
In [64]: # 8
# Случайные 3D точки
n = 100
x = rand(n)
y = rand(n)
z = rand(n)

p = scatter(x, y, z,
            title="3D точечный график случайных данных",
            xlabel="x", ylabel="y", zlabel="z",
            label="Случайные точки",
            color=:red, markersize=5,
            legend=:bottomleft,
            camera=(30, 30)) # угол обзора
```

Out[64]: 3D точечный график случайных данных

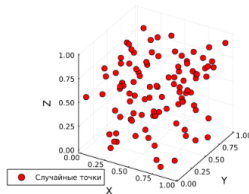


Рис. 61: 3-мерный точечный график случайных данных

9. Создайте анимацию с построением синусоиды. То есть вы строите последовательность графиков синусоиды, постепенно увеличивая значение аргумента. После соедините их в анимацию

# Задания для самостоятельного выполнения

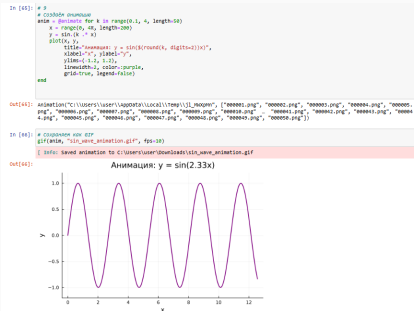


Рис. 62: Анимация построения синусоиды

10. Постройте анимированную гипоциклоиду для 2 целых значений модуля  $k$  и 2 рациональных значений модуля  $k$

## Задания для самостоятельного выполнения

```
[47]: # 10
function hypocycloid(t, k)
    k = k * pi/360; %преобразуем градусы в радианы
    r = 1 - k; %радиус вписанного
    x = (R - r) * cos(t) + r * cos((R - r)/r * t)
    y = (R - r) * sin(t) - r * sin((R - r)/r * t)
    return x, y
end

% Выводим значения R
k_values = [3, 5, 3/2, 5/3]; % 2 случая, 2 рациональных

% Анимация: будем показывать по одному k за раз
anim = @(k) for k in k_values
    t = range(0, 2*pi, length(k)/100)
    x, y = hypocycloid(t, k)

    plot(x, y,
        title="Гипоциклоида, k = %k",
        xlabel="x", ylabel="y",
        aspect="equal",
        linewidth, color="darkorange",
        grid=true, legend=false,
        xlim=(-max(abs(x))..."^1.1, max(abs(x))..."^1.1),
        ylim=(-max(abs(y))..."^1.1, max(abs(y))..."^1.1))
    end

Out[47]: Animation("C:\Users\user\AppData\Local\Temp\jj_fik3ag", ["000001.png", "000002.png", "000003.png", "000004.png"])

In [48]: gif(anim, "hypocycloid_animation.gif", fps=3)

[ Info: Saved animation to C:\Users\user\Downloads\hypocycloid_animation.gif

Out[48]: Гипоциклоида, k = 3/1]
```

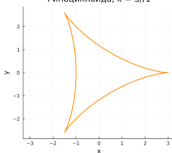


Рис. 63: Анимация гипоциклоиды

11. Постройте анимированную эпициклоиду для 2 целых значений модуля  $k$  и 2 рациональных значений модуля  $k$

# Задания для самостоятельного выполнения

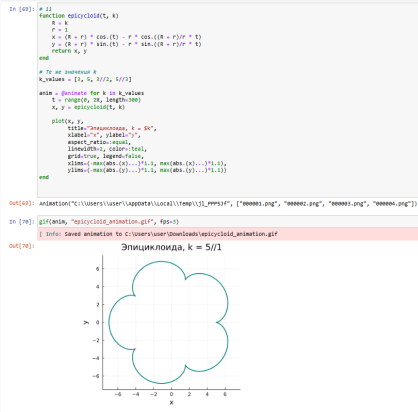


Рис. 64: Анимация эпициклоида

## Выводы

---

В результате выполнения данной лабораторной работы я освоила синтаксис языка Julia для построения графиков.